proc.c M

kernel > proc.c

```c

// Create a new thread sharing the calling process's address space.
// fn    - user function the thread will execute
// arg   - argument passed to fn (in a0)
// stack - top of the pre-allocated user stack (stack grows downward)
// Returns thread pid on success, -1 on failure.
int
kclone(uint64 fn, uint64 arg, uint64 stack)
{
  int i, pid;
  struct proc *nt;            // new thread
  struct proc *p = myproc();  // calling process

  // Allocate a proc slot (gets its own kernel stack, trapframe, context).
  if((nt = allocproc()) == 0){
    return -1;
  }

  // Create a lightweight page table: maps trampoline + trapframe,
  // then shares user pages from the parent (same physical pages).
  // proc_pagetable already maps TRAMPOLINE and TRAPFRAME for nt.
  // (allocproc already called proc_pagetable for nt — we re-use it)

  // Share user-space pages into the thread's page table.
  if(uvmshare(p->pagetable, nt->pagetable, p->sz) < 0){
    freeproc(nt);
    release(&nt->lock);
    return -1;
  }
  nt->sz = p->sz;

  // Copy the calling process's trap frame so the thread inherits
  // open file state, program counter layout, etc.
  *(nt->trapframe) = *(p->trapframe);

  // Override epc, a0, sp to start the thread at fn(arg) on its own stack.
  nt->trapframe->epc = fn;    // thread entry point
  nt->trapframe->a0  = arg;   // first argument
  nt->trapframe->sp  = stack; // pre-allocated user stack top

  // Share open file descriptors.
  for(i = 0; i < NOFILE; i++)
    if(p->ofile[i])
      nt->ofile[i] = filedup(p->ofile[i]);
  nt->cwd = idup(p->cwd);

  safestrcpy(nt->name, p->name, sizeof(p->name));

  // Mark as thread so freeproc skips freeing shared pages.
  nt->is_thread = 1;

  pid = nt->pid;

  release(&nt->lock);

  acquire(&wait_lock);
  nt->parent = p;
  release(&wait_lock);

  acquire(&nt->lock);
  nt->state = RUNNABLE;
  release(&nt->lock);

  return pid;
}
```

proc.c M

kernel > proc.c

```c
// Wait for any child THREAD (is_thread==1) to exit.
// Returns the exited thread's pid, or -1 if no thread children.
int
kjoin(void)
{
  struct proc *pp;
  int havekids, pid;
  struct proc *p = myproc();

  acquire(&wait_lock);

  for(;;){
    havekids = 0;
    for(pp = proc; pp < &proc[NPROC]; pp++){
      if(pp->parent == p && pp->is_thread){
        acquire(&pp->lock);
        havekids = 1;
        if(pp->state == ZOMBIE){
          pid = pp->pid;
          freeproc(pp);
          release(&pp->lock);
          release(&wait_lock);
          return pid;
        }
        release(&pp->lock);
      }
    }

    if(!havekids || killed(p)){
      release(&wait_lock);
      return -1;
    }

    // Sleep until a thread child exits.
    sleep(p, &wait_lock);
  }
}
```

proc.c M

kernel > proc.c

```c
}

// Free a thread's page table structure.
// Does NOT free the user physical pages (shared with parent).
void
proc_freepagetable_thread(pagetable_t pagetable, uint64 sz)
{
  uvmunmap(pagetable, TRAMPOLINE, 1, 0);
  uvmunmap(pagetable, TRAPFRAME, 1, 0);
  // Unmap shared user pages without freeing the physical memory.
  if(sz > 0)
    uvmunmap(pagetable, 0, PGROUNDUP(sz)/PGSIZE, 0);
  freewalk(pagetable);
}
```

proc.c Git Local Changes (Working Tree) - 1 of 4 changes

```
  if(p->trapframe)
    kfree((void*)p->trapframe);
  p->trapframe = 0;
  if(p->pagetable)
    proc_freepagetable(p->pagetable, p->sz);
  if(p->pagetable) {
    if(p->is_thread)
      // Thread: free page table structure only, not shared physical pages.
      proc_freepagetable_thread(p->pagetable, p->sz);
    else
      proc_freepagetable(p->pagetable, p->sz);
  }
```

vm.c M

kernel > vm.c

```c

// Share user memory pages from parent into child's page table (for threads).
// Copies PTEs without allocating new physical memory - threads share pages.
// On failure, unmaps what was mapped (without freeing physical pages).
// returns 0 on success, -1 on failure.
int
uvmshare(pagetable_t old, pagetable_t new, uint64 sz)
{
  pte_t *pte;
  uint64 pa, i;
  uint flags;

  for(i = 0; i < sz; i += PGSIZE){
    if((pte = walk(old, i, 0)) == 0)
      continue;   // page table entry not allocated
    if((*pte & PTE_V) == 0)
      continue;   // physical page not allocated
    pa = PTE2PA(*pte);
    flags = PTE_FLAGS(*pte);
    // Map the same physical page into the new page table (no alloc).
    if(mappages(new, i, PGSIZE, pa, flags) != 0){
      goto err;
    }
  }
  return 0;

 err:
  // Unmap what we mapped, but do NOT free the physical pages (they belong to parent).
  uvmunmap(new, 0, i / PGSIZE, 0);
  return -1;
}

```

usys.pl M | syscall.c M | syscall.h M | Review Changes | ulib.c M | Walkthrou

user > ulib.c

```c
// thread_create allocates a page-sized stack, calls clone(fn, arg, stacktop).
// Returns the new thread's pid, or -1 on error.
// The stack is passed back via *stack so the caller can free it after join.
int
thread_create(void(*fn)(void*), void *arg, void **stack)
{
  char *s = malloc(PGSIZE);
  if(s == 0)
    return -1;
  *stack = s;
  // Stack grows downward- pass the top of the allocated region.
  int pid = clone(fn, arg, s + PGSIZE);
  if(pid < 0){
    free(s);
    *stack = 0;
    return -1;
  }
  return pid;
}

// Wait for any child thread to exit.
// If stack is non-null, frees the memory pointed to by *stack and clears it.
// Returns the joined thread's pid, or -1 Son error.
int
thread_join(void **stack)
{
  int pid = join();
  if(pid < 0)
    return -1;
  if(stack && *stack){
    free(*stack);
    *stack = 0;
  }
  return pid;
}
```

```c
[SYS_clone]        sys_clone,
[SYS_join]         sys_join,
```

```c
extern uint64 sys_clone(void);
extern uint64 sys_join(void);
```

```c
#define SYS_clone        27
#define SYS_join         28
```

h defs.h M ×

kernel > h defs.h

```
// proc.c
int             cpuid(void);
void            kexit(int);
int             kfork(void);
int             kclone(uint64, uint64, uint64);
int             kjoin(void);
int             growproc(int);
void            proc_mapstacks(pagetable_t);
pagetable_t     proc_pagetable(struct proc *);
void            proc_freepagetable(pagetable_t, uint64);
void            proc_freepagetable_thread(pagetable_t, uint64);
int             kkill(int);
int             killed(struct proc*);
```

h defs.h M ×

kernel > h defs.h

```
// vm.c
void            kvminit(void);
void            kvminithart(void);
void            kvmmap(pagetable_t, uint64, uint64, uint64, int);
int             mappages(pagetable_t, uint64, uint64, uint64, int);
pagetable_t     uvmcreate(void);
uint64          uvmalloc(pagetable_t, uint64, uint64, int);
uint64          uvmdealloc(pagetable_t, uint64, uint64);
int             uvmcopy(pagetable_t, pagetable_t, uint64);
int             uvmshare(pagetable_t, pagetable_t, uint64);
void            freewalk(pagetable_t);
void            uvmfree(pagetable_t, uint64);
void            uvmunmap(pagetable_t, uint64, uint64, int);
void            uvmclear(pagetable_t, uint64);
```

usys.S M ×

user > usys.S

```
lock_create:
.global lock_acquire
lock_acquire:
 li a7, SYS_lock_acquire
 ecall
 ret
.global lock_release
lock_release:
 li a7, SYS_lock_release
 ecall
 ret
.global lock_destroy
lock_destroy:
 li a7, SYS_lock_destroy
 ecall
 ret
.global clone
clone:
 li a7, SYS_clone
 ecall
 ret
.global join
join:
 li a7, SYS_join
 ecall
 ret
```

```
entry("lock_destroy");
entry("clone");
entry("join");
```

h user.h M ×

user > h user.h

```
void* memset(void*, int, uint);
int atoi(const char*);
int memcmp(const void *, const void *, uint);
void *memcpy(void *, const void *, uint);
char* sbrk(int);
char* sbrklazy(int);
int   thread_create(void(*fn)(void*), void *arg, void **stack);
int   thread_join(void **stack);

// printf.c
void fprintf(int, const char*, ...) __attribute__ ((format (printf, 2, 3)));
void printf(const char*, ...) __attribute__ ((format (printf, 1, 2)));

// umalloc.c
void* malloc(uint);
void free(void*);
```